3. Business Scenario

Modern SIEM platforms receive millions of logs every day from servers, applications, firewalls, network devices, cloud platforms, and security tools.

While these logs provide valuable operational information, manually understanding thousands of log entries is time-consuming and inefficient.

Your task is to build an AI-powered Log Intelligence Engine capable of understanding Apache server logs and exposing intelligent REST APIs that assist security analysts and system administrators.

Although the supplied dataset contains Apache logs, your solution should be architected such that additional log formats could be supported in the future without requiring significant architectural changes.

---

4. Dataset

The following Apache log dataset will be provided as part of the project.

Dataset Reference

https://github.com/logpai/loghub/blob/master/Apache/Apache_2k.log

The log file will already exist inside the project directory.

Candidates are free to preprocess the dataset into JSON, CSV or any other structured format before analysis.

---

5. Objective

Develop an AI-powered Log Intelligence Engine capable of providing intelligent insights through REST APIs.

The system should demonstrate:

- Log preprocessing
- AI reasoning
- Intelligent categorization
- Timeline generation
- Root cause analysis
- Clean software architecture
- Efficient API design

---

6. Technical Requirements

Candidates are free to choose their preferred libraries, frameworks, prompting strategies and AI architecture.

The following requirements must be satisfied.

Backend

- Node.js
- Express.js
- REST APIs

Frontend

A minimal interface that demonstrates all implemented APIs.

Examples include:

- Upload log file
- Execute APIs
- View responses

UI design is **not** part of the evaluation.

Database

No database should be used.

The application should process logs in memory.

AI

Candidates may use

- OpenAI
- Gemini
- Ollama
- Hugging Face
- Open-source LLMs
- LangChain
- LlamaIndex
- Haystack
- Any other suitable AI framework

Important Constraint

The entire log file **must not** be sent to the LLM for every API request.

Candidates are expected to design an efficient preprocessing and context-selection strategy before interacting with the language model.

---

7. Feature 1 - AI Log Classification Engine

Objective

Classify Apache log entries into meaningful operational categories.

The classifier should process one or more log entries and return structured classifications.

Suggested Categories

- Startup
- Shutdown
- Configuration
- Worker Initialization

- Backend Communication
- Warning
- Error
- Performance
- Security
- Unknown

Candidates may introduce additional categories where appropriate.

API

POST

/api/ai/log-classification

Expected Request

The API may accept either

- A single log entry

or

- Multiple log entries

The request format is left to the candidate.

Expected Response

Each log should include

- Category
- Confidence Score
- AI Explanation

Example

- Category: Worker Initialization
- Confidence: 96%

- Explanation: Apache worker environment initialized successfully.

---

8. Feature 2 - AI Incident Timeline Generator

Objective

Generate an incident timeline from multiple log entries.

The timeline should identify important operational events and present them chronologically.

The AI should intelligently summarize related logs into meaningful timeline entries instead of simply displaying raw logs.

API

POST

/api/ai/incident-timeline

Expected Output

The timeline should include

- Timestamp
- Event Title
- Summary
- Supporting Log References

Example

10:02

Apache Server Started

Apache initialization completed successfully.

10:05

Worker Environment Initialized

Workers loaded successfully.

10:18

Backend Communication Failure

Repeated communication failures observed between Apache and backend service.

---

9. Feature 3 - AI Root Cause Analysis

Objective

Analyze a collection of related logs and determine the most probable root cause of an incident.

The response should be evidence-driven and supported by relevant log entries.

API

POST

/api/ai/root-cause-analysis

Expected Response

The response should include

- Root Cause
- Supporting Evidence
- Impact
- Recommended Action
- Confidence Score

Example

Root Cause

Backend Tomcat service unavailable.

Evidence

Repeated worker failures

Backend communication timeout

Connection retries

Impact

Users may experience application downtime.

Recommendation

Verify backend service availability and restart Apache workers if necessary.

Confidence

91%

---

10. Common API Response Format

All APIs should follow a consistent response format.

Example

```
{
   "success": true,
   "message": "Timeline generated successfully",
   "processingTimeMs": 842,
   "data": {}
}
```

---

11. Performance Expectations

The solution should demonstrate thoughtful engineering practices.

Examples include

- Efficient log parsing
- Reusable preprocessing
- Avoid repeated parsing
- Efficient AI context generation
- Modular architecture

Although the provided dataset is relatively small, the architecture should demonstrate how it could scale for significantly larger datasets in a production SIEM environment.

---

12. User Interface

Develop a minimal UI capable of demonstrating the implemented APIs.

The interface may include

- Load dataset
- Execute API
- Display AI responses

The visual appearance of the UI will not be evaluated.

---

13. Deliverables

Candidates must submit

- Complete source code
- Deployment URL
- 5 - 10 minute demonstration video
- README containing setup instructions
- Architecture document describing

- AI workflow
- Preprocessing strategy
- Prompt engineering approach
- Overall architecture

---

14. Evaluation Criteria

Criteria	Weight
AI Capability & Accuracy	45%
Performance & Scalability	25%
Code Quality & Architecture	20%
API Design & Overall Completeness	10%

---

15. Acceptance Criteria

A submission will be considered complete if

- All three APIs are fully functional.
- The application successfully processes the provided Apache log dataset.
- AI-generated outputs are relevant, meaningful, and supported by the supplied log data where applicable.
- The application is deployed and accessible through the submitted deployment URL.
- The codebase is modular, readable, and maintainable.
- The demonstration video clearly showcases the implemented functionality.

---

16. Notes

- Candidates are encouraged to make reasonable engineering decisions where requirements are intentionally left open.
- Innovation and thoughtful architecture are valued over unnecessary complexity.
- The use of external AI services, frameworks, and libraries is permitted.